```
import random
import time

def random_board(n):
    return [random.randint(0, n - 1) for _ in range(n)]

def count_attacks(board):
    n = len(board)
    attacks = 0
    for i in range(n):
        for j in range(i + 1, n):
            if board[i] == board[j]:
                attacks += 1
            if abs(board[i] - board[j]) == abs(i - j):
                attacks += 1
    return attacks

def get_best_neighbour(board):
    n = len(board)
    best = board[:]
    best_attacks = count_attacks(board)
    for row in range(n):
        for col in range(n):
            if col == board[row]:
                continue
            neighbour = board[:]
            neighbour[row] = col
            attacks = count_attacks(neighbour)
            if attacks < best_attacks:
                best_attacks = attacks
                best = neighbour[:]
    return best, best_attacks

def hill_climbing_with_steps(n, max_restarts=1000):
    all_restarts = []
    for restart in range(max_restarts):
        board = random_board(n)
        steps = [board[:]]
        while True:
            current_attacks = count_attacks(board)
            if current_attacks == 0:
                return board, steps, restart, all_restarts
            neighbour, neighbour_attacks = get_best_neighbour(board)
            if neighbour_attacks >= current_attacks:
                all_restarts.append(steps)
                break
            board = neighbour
            steps.append(board[:])
    return None, [], max_restarts, all_restarts

def attacking_rows(board):
    n = len(board)
    bad = set()
    for i in range(n):
        for j in range(i + 1, n):
```

```
        for j in range(i + 1, n):
            if board[i] == board[j] or abs(board[i] - board[j]) == abs(i - j):
                bad.add(i)
                bad.add(j)
    return bad

print('Core algorithm loaded.')
```

```
def print_board(board, label=''):
    n = len(board)
    if label:
        print(label)
    line = '+' + ('---+' * n)
    bad = attacking_rows(board)
    for row in range(n):
        print(line)
        row_str = '|'
        for col in range(n):
            if board[row] == col:
                row_str += ' Q |' if row not in bad else ' X |'
            else:
                row_str += ' . |'
        print(row_str)
    print(line)
    print(f'Attacks: {count_attacks(board)}')

def run_cmd_demo(n):
    print()
    print('N-QUEENS PROBLEM - Hill Climbing Algorithm')
    print(f'N = {n}')
    print()

    initial = random_board(n)
    print('INITIAL PROBLEM (random placement):')
    print_board(initial)
    print()

    t_start = time.time()
    solution, steps, restarts, failed_runs = hill_climbing_with_steps(n)
    t_end = time.time()
    elapsed = t_end - t_start

    if solution is None:
        print('No solution found.')
        return None, [], 0, 0

    print(f'Solving... took {len(steps)-1} steps across {restarts} restart(s).')
    print()

    for i, board in enumerate(steps):
        if i == 0:
            print('Step 0 - Initial state (this run):')
        elif i == len(steps) - 1:
            print(f'Step {i} - SOLUTION:')
        else:
            print(f'Step {i}:')
```

```
        print_board(board)
        print()

    print('SOLUTION FOUND')
    print(f'Board        : {solution}')
    print(f'Attacks      : {count_attacks(solution)}')
    print(f'Steps taken  : {len(steps) - 1}')
    print(f'Restarts     : {restarts}')
    print(f'Time taken   : {elapsed:.4f} seconds')
    print(f'Time complex.: O(n^3) per step, O(restarts * steps * n^3) total')
    print(f'Space complex: O(n)')

    return solution, steps, restarts, elapsed

print('CMD demo function loaded.')
```

```
valid = [4, 8, 10, 12, 16, 20]

while True:
    try:
        N = int(input(f'Enter N {valid}: '))
        if N in valid:
            break
        print(f'Please enter one of {valid}')
    except ValueError:
        print('Invalid input. Enter a number.')

solution, steps, restarts, elapsed = run_cmd_demo(N)
```

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from IPython.display import display, clear_output
import ipywidgets as widgets

LIGHT  = '#F0D9B5'
DARK   = '#B58863'
ATTACK = '#e63946'
SOLVE  = '#2a9d8f'

def draw_board_mpl(board, ax, title=''):
    n = len(board)
    ax.clear()
    ax.set_xlim(0, n)
    ax.set_ylim(0, n)
    ax.set_aspect('equal')
    ax.axis('off')
    ax.set_title(title, fontsize=13, fontweight='bold', pad=10,
                 color='white', fontfamily='monospace')

    bad = attacking_rows(board)
    attacks = count_attacks(board)
    is_solution = attacks == 0

    for row in range(n):
        for col in range(n):
```

```
            brow = n - 1 - row
            if board[brow] == col and brow in bad:
                color = ATTACK
            elif board[brow] == col and is_solution:
                color = SOLVE
            elif (row + col) % 2 == 0:
                color = LIGHT
            else:
                color = DARK

            rect = patches.Rectangle((col, row), 1, 1,
                                     linewidth=0.5, edgecolor='black', facecolor=color)
            ax.add_patch(rect)

            if board[brow] == col:
                ax.text(col + 0.5, row + 0.5, 'Q',
                        ha='center', va='center',
                        fontsize=max(6, int(180 / n)),
                        fontweight='bold', color='#1a1a2e',
                        fontfamily='monospace')

    info = f'Attacks: {attacks}'
    ax.text(n / 2, -0.6, info, ha='center', va='center',
            fontsize=10, color='#f9e2af', fontfamily='monospace')

print('Visualization functions loaded.')
```

```
if solution:
    fig, ax = plt.subplots(figsize=(6, 6))
    fig.patch.set_facecolor('#1e1e2e')
    ax.set_facecolor('#1e1e2e')
    draw_board_mpl(steps[0], ax, title=f'INITIAL PROBLEM  (N={N})')
    plt.tight_layout()
    plt.show()
```

```
if solution:
    total = len(steps)
    cols = min(4, total)
    rows = (total + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(cols * 4, rows * 4))
    fig.patch.set_facecolor('#1e1e2e')

    if total == 1:
        axes = [[axes]]
    elif rows == 1:
        axes = [axes]

    for idx, board in enumerate(steps):
        r, c = divmod(idx, cols)
        ax = axes[r][c]
        ax.set_facecolor('#1e1e2e')
        if idx == 0:
            title = 'Step 0: Initial'
        elif idx == total - 1:
            title = f'Step {idx}: SOLUTION'
```

```
        else:
            title = f'Step {idx}'
        draw_board_mpl(board, ax, title=title)

    for idx in range(total, rows * cols):
        r, c = divmod(idx, cols)
        axes[r][c].axis('off')
        axes[r][c].set_facecolor('#1e1e2e')

    plt.suptitle(f'N={N}  |  {total-1} steps  |  {restarts} restart(s)  |  {elapsed:.4f}s',
                 color='white', fontsize=12, fontfamily='monospace', y=1.01)
    plt.tight_layout()
    plt.show()
```

```
if solution:
    total = len(steps)

    step_slider = widgets.IntSlider(
        value=0, min=0, max=total - 1, step=1,
        description='Step:',
        style={'description_width': 'initial'},
        layout=widgets.Layout(width='500px')
    )

    play_btn   = widgets.Play(value=0, min=0, max=total - 1, step=1,
                              interval=700, description='Play')
    widgets.jslink((play_btn, 'value'), (step_slider, 'value'))

    out = widgets.Output()

    def on_step_change(change):
        idx = change['new']
        board = steps[idx]
        atk = count_attacks(board)
        if idx == 0:
            title = f'Step 0: Initial Problem  |  Attacks: {atk}'
        elif idx == total - 1:
            title = f'Step {idx}: SOLUTION FOUND  |  Attacks: {atk}'
        else:
            title = f'Step {idx} of {total-1}  |  Attacks: {atk}'

        with out:
            clear_output(wait=True)
            fig, ax = plt.subplots(figsize=(6, 6))
            fig.patch.set_facecolor('#1e1e2e')
            ax.set_facecolor('#1e1e2e')
            draw_board_mpl(board, ax, title=title)
            plt.tight_layout()
            plt.show()

    step_slider.observe(on_step_change, names='value')

    stats = widgets.HTML(
        value=f'<span style="font-family:monospace; color:#cdd6f4">'
              f'N={N}  |  Steps={total-1}  |  '
              f'Restarts={restarts}  |  Time={elapsed:.4f}s  |  '
```

```
                f'Time O(): O(n^3/step)  |  Space O(): O(n)</span>'
        )

        display(stats)
        display(widgets.HBox([play_btn, step_slider]))
        display(out)

        on_step_change({'new': 0})
```

```
    if solution:
        fig, ax = plt.subplots(figsize=(6, 6))
        fig.patch.set_facecolor('#1e1e2e')
        ax.set_facecolor('#1e1e2e')
        draw_board_mpl(solution, ax,
                       title=f'SOLUTION  |  N={N}  |  Steps={len(steps)-1}  |  Restarts={restarts}')
        plt.tight_layout()
        plt.show()

        print(f'Board        : {solution}')
        print(f'Attacks      : {count_attacks(solution)}')
        print(f'Steps taken  : {len(steps) - 1}')
        print(f'Restarts     : {restarts}')
        print(f'Time taken   : {elapsed:.4f} seconds')
        print(f'Time complex.: O(n^3) per step')
        print(f'Space complex: O(n)')
```